

Custom CPQ on Salesforce

Solution Design

DOCUMENT CONTROL

DOCUMENT ID	LP-REVTECH-CPQ-SD-001
VERSION	1.0
STATUS	For review
AUTHOR	Bruno Romano
DATE	7 September 2026
AUDIENCE	RevTech engineering · RevOps · Platform engineering
COMPANION ARTEFACT	Apex prototype and test suite (repository)
SCOPE	Catalog exposure, price-floor enforcement, commit to Opportunity, Closed Won entitlement automation, logging

ABSTRACT

This document specifies the Salesforce-side layer behind the CPQ experience being built on Luxury Presence's internal React/NestJS platform. It covers the data model, the externally-callable integration surface, the security posture on that surface, the Apex automation that produces durable entitlements at Closed Won, and the logging approach. It does not specify the client application, which is out of scope. Every design decision that is not self-evident is called out and numbered **D-n** so it can be referenced in review.

Contents

1	Scope and the governing constraint	3
2	Assumptions, exclusions and open questions	4
3	Business rules that become code	5
4	Data model	7
4.1	Entity-relationship diagram	7
4.2	Catalog item — everything sellable, in one object	8
4.6	Why Product2 exists alongside a custom catalog	9
5	Entitlement aggregation	10
6	Integration surface	11
6.3	Status codes and error contract	12
6.4	Authentication and rep identity	13
7	Security and observability	14
8	Closed Won automation	15
9	Deferred scope	16
10	Test strategy	16
A	Appendix — Catalog reference data	17

1 Scope and the governing constraint

Reps configure deals in the React/NestJS platform. Salesforce is the catalog of record, the authority that enforces commercial rules, and the origin of durable entitlements. No one opens a Salesforce screen in this workflow.

That produces a single constraint from which everything else follows.

GOVERNING CONSTRAINT

Salesforce cannot trust the caller. Every rule in this document must hold when the request comes from a script, a Postman tab, or a bug in the platform — not only from the intended UI.

Three consequences run through the whole design. Validation is re-run inside commit, and a prior `validate` call is never trusted. The price floor lives in Apex and is never returned as advisory data the client is expected to honour. The API surface is typed and explicitly versioned, because a client outside this repository will build against it and cannot be redeployed by us.

Everything that follows is either a consequence of that constraint, or a decision about where a rule should live so that it stays enforceable.

2 Assumptions, exclusions and open questions

2.1 Assumptions

ID	ASSUMPTION	CONSEQUENCE IF WRONG
A-1	Add-on prices, floors and charge types are invented. Design Refresh \$750 / floor \$600 / one-time; Competitor Analysis \$150 / floor \$120 / recurring; Contacts (500) \$100 / floor \$80 / recurring.	None structural. The brief names the three add-ons but not their commercials; these live in catalog records that RevOps edits without a deployment.
A-2	Plan prices are monthly recurring; setup fees are one-time; billing frequency is monthly.	Changes the recurring vs. one-time split that billing and entitlement derivation both key off.
A-3	USD only.	Multi-currency makes the catalog a per-currency price row. Not precluded by the model.
A-4	The Opportunity has an Account before commit.	Entitlements are held against a customer; an Opportunity without one cannot produce them, so commit rejects it.
A-5	Permitted contract terms are 12/24/36 months, held as configuration rather than a hard-coded list.	None. Sales will change this, and the design expects it to.
A-6	The setup fee is discountable to \$0 but never above the catalog amount.	A ceiling rule rather than a floor rule. See Q-3.
A-7	One active Subscription per Account at a time.	Overlapping subscriptions become a renewals concern; deferred to §9.
A-8	The platform holds its own credentials and passes the rep's identity in the request body. Reps never hold Salesforce credentials.	Without this, no discount is attributable to a person. See §6.4.

2.2 Out of scope

No UI or front-end build. No live OAuth handshake — the auth pattern is specified in §6.4 but not implemented. No Opportunity creation; the interface accepts an existing Id. No live callouts to the feature-gating or billing systems. No approval workflow for sub-floor pricing; a hard pass/fail is the requirement. No use of `Quote` or `QuoteLineItem` — contracts are generated in PandaDoc, so `OpportunityLineItem` is the source of truth for what is being sold. Renewals, amendments, co-terming, proration and cancellation are deferred to §9; the model is shaped not to preclude them.

2.3 Open questions

ID	QUESTION	OWNER	STATUS PENDING RESOLUTION
Q-1	Is uplift above list price permitted, or is there a ceiling as well as a floor?	Sales leadership	Currently unbounded. A ceiling is one rule in the same validator.
Q-2	Confirm add-on prices, floors and charge types (A-1).	RevOps	Placeholder catalog data. No code impact.
Q-3	Can any rep waive the setup fee to \$0, or does the waiver need its own floor or an approval?	Sales / Finance	Currently freely waivable — the discount lever most likely to be leaned on once the plan floor is enforced.
Q-4	When a contract term ends, who deactivates entitlements — a scheduled Salesforce job, or the gating service reading the end date?	RevTech engineering	The model supports both; every entitlement carries an active window. Operational owner undecided.

3 Business rules that become code

Only the rules a test asserts against. Each maps to a test method in §10.

CATALOG

- R-1 Catalog items carry a stable, human-readable code (`PLAN_BRAND` , `ADDON_CONTACTS_500`). Downstream systems key on the code, never on the Salesforce Id.
- R-2 Only items active on the configuration's effective date may be quoted.
- R-3 Changing a catalog price or floor never retroactively alters committed lines or existing entitlements.

CONFIGURATION

- R-4 Exactly one plan. Zero or multiple is invalid.
- R-5 An add-on appears at most once; stacking is expressed as quantity, not repeated lines. Quantity is a positive integer.
- R-6 Add-ons that duplicate a plan-included feature are valid in both directions, by design.

PRICING

- R-7 Negotiated plan price $\geq$ that plan's floor. Inclusive: exactly at floor passes.
- R-8 Each add-on has its own floor; a negotiated price below it fails.
- R-9 Setup fee may be discounted to \$0 but may not exceed the catalog amount.
- R-10 Prices are USD, non-negative, two decimal places.
- R-11 Validation returns every violation in one response. A rep fixing three lines should make one round trip.

CONTRACT

- R-12 Effective date required; term must be one of the configured permitted values.

COMMIT

- R-13 Commit re-runs full validation server-side. A prior successful validate is never trusted.
- R-14 Commit is atomic. All lines and the Opportunity term fields, or nothing.
- R-15 Commit is refused if the target Opportunity is already Closed. Closed deals are immutable to this API.
- R-16 Re-committing replaces the lines this API previously wrote, and leaves lines added by other means untouched.
- R-17 Commit is idempotent by key: the same request twice yields one set of lines and returns the original result.

ENTITLEMENTS

- R-18 Entitlement quantity per feature is the sum of the plan-included quantity and matching add-on quantities. Binary features do not accumulate; metered features do.
- R-19 Only recurring, feature-bearing items produce entitlements. The setup fee and Design Refresh are billable and grant nothing.
- R-20 Entitlement creation is idempotent and bulk-safe. A 200-record stage update, or an already-won Opportunity re-saved, creates no duplicates.
- R-21 Entitlements are durable: superseded, never silently overwritten.

4 Data model

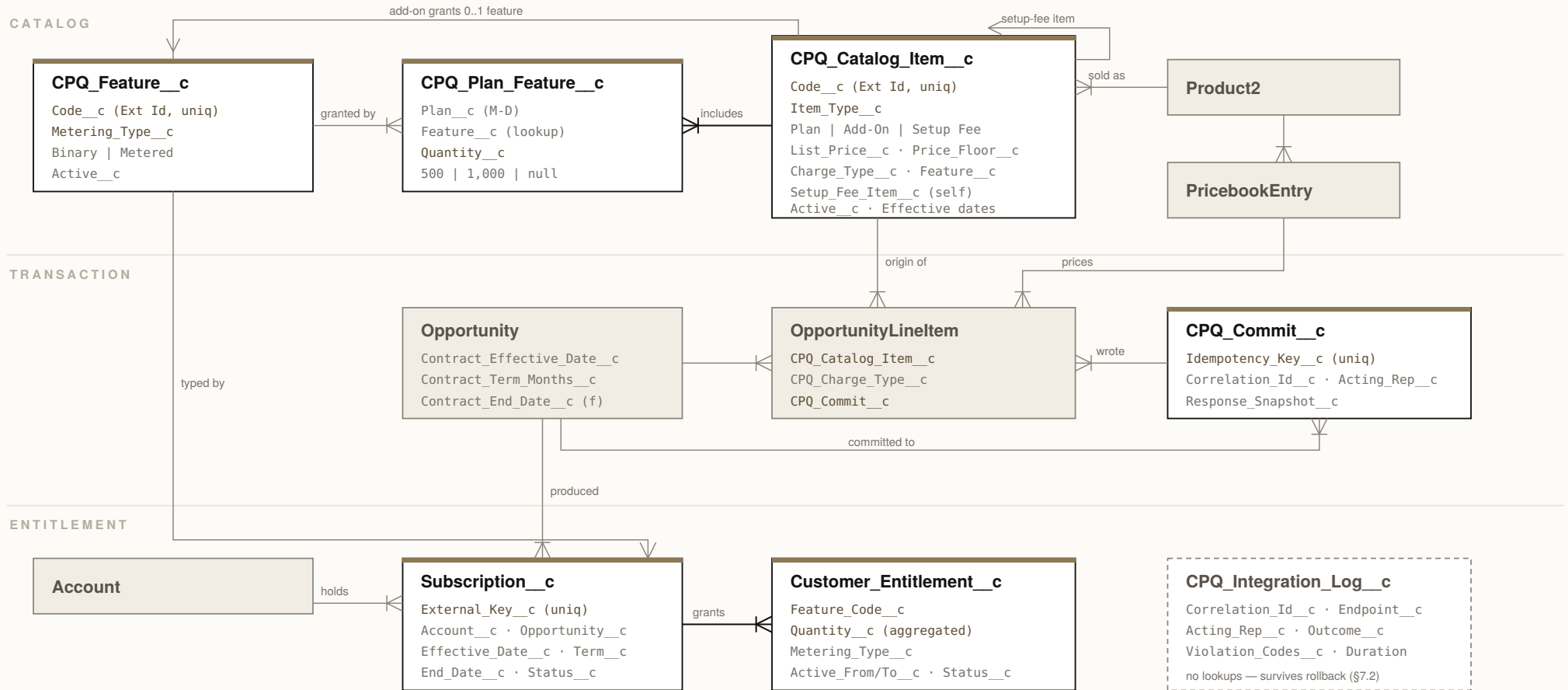


FIGURE 1 Entity-relationship diagram. Custom objects built by this project carry a brass rule; standard Salesforce objects are shaded.

■ custom object ■ standard object — master-detail — lookup no relationship, by design

4 Data model — object specifications

4.2 CPQ_Catalog_Item__c — everything sellable, in one object

Plans, add-ons and setup fees share a shape: a code, a list price, a floor, an active window and a Product2 .

FIELD	TYPE	NOTE
Code__c	Text · External Id · Unique	PLAN_BRAND , ADDON_CONTACTS_500 , SETUP_BRAND
Item_Type__c	Picklist	Plan / Add-On / Setup Fee
List_Price__c , Price_Floor__c	Currency (16,2)	—
Charge_Type__c	Picklist	Recurring Monthly / One-Time
Product__c	Lookup (Product2)	Required. See §4.6
Setup_Fee_Item__c	Lookup (self)	Plans only → the Setup Fee item
Feature__c	Lookup (CPQ_Feature__c)	Add-ons only. Null for Design Refresh
Feature_Quantity_Per_Unit__c	Number	Contacts (500) → 500
Active__c , Effective_Start__c , Effective_End__c	Checkbox, Date, Date	Enforces R-2

D-1 DESIGN DECISION — ONE CATALOG OBJECT, NOT THREE

Modelling setup fees as catalog items rather than as a number on the plan makes every committed line uniform: one catalog item → one Product2 → one OpportunityLineItem . There is no second code path for the setup fee, and RevOps can reprice one without touching plan records.

4.3 CPQ_Feature__c — the capability catalog

Kept deliberately separate from the product catalog. Products are what we sell; features are what a customer is entitled to. Competitor Analysis is both, and conflating the two is precisely what makes the overlap case hard.

FIELD	TYPE	NOTE
Code__c	Text · External Id · Unique	FEAT_CONTACTS
Metering_Type__c	Picklist	Binary / Metered — the field that decides whether quantities accumulate
Active__c	Checkbox	—

4.4 Junction, commit and transaction objects

`CPQ_Plan_Feature__c` — master-detail to the plan, lookup to the feature, plus `Quantity__c`. Plan inclusion carries a quantity (All In → `FEAT_CONTACTS` × 1,000), so it cannot be a checkbox.

`CPQ_Commit__c` — one row per commit attempt, holding `Correlation_Id__c`, `Acting_Rep__c`, `Opportunity__c`, `Status__c` and `Response_Snapshot__c`, the original response replayed verbatim on a duplicate key.

D-2 DESIGN DECISION — IDEMPOTENCY ENFORCED BY THE DATABASE

`Idempotency_Key__c` is a unique External Id. That is what makes R-17 real rather than aspirational: uniqueness is enforced by the platform, so two concurrent identical requests cannot both win.

Querying for an existing key and then inserting would pass every test I could write and still lose a race in production.

`OpportunityLineItem` takes three custom fields. `CPQ_Catalog_Item__c` is a durable link back to the catalog, so the Closed Won automation never re-derives anything by name matching.

`CPQ_Charge_Type__c` carries the billing distinction. `CPQ_Commit__c` is how R-16 works: replacing a configuration deletes only lines stamped by a prior commit of this API, leaving anything a human or another process added alone.

`Opportunity` takes `Contract_Effective_Date__c`, `Contract_Term_Months__c` and `Contract_End_Date__c` (formula).

4.5 Entitlement objects

`Subscription__c` — Account, Opportunity, plan, effective date, term, end date, status. `External_Key__c` is a unique External Id set to the Opportunity Id, which is what makes the Closed Won automation idempotent under `upsert`.

`Customer_Entitlement__c` — named to avoid colliding with the standard Service Cloud `Entitlement` object, which is not used here. Master-detail to Subscription, lookup to Feature, plus `Feature_Code__c` (denormalised so downstream never has to join), `Quantity__c` (already aggregated), `Metering_Type__c`, `Active_From__c` / `Active_To__c` and `Status__c`.

4.6 Why `Product2` exists alongside a custom catalog

`OpportunityLineItem` cannot be inserted without a `PricebookEntryId`. That is a platform constraint, not a preference. Each sellable catalog item therefore maps to a `Product2` with a standard `PricebookEntry`.

D-3 DESIGN DECISION — DIVISION OF RESPONSIBILITY

`Product2` / `PricebookEntry` is the commercial record the platform requires. `CPQ_Catalog_Item__c` is the source of truth for the CPQ rules — floors, included features, charge type, effective windows — none of which `Product2` can express.

Putting floors in a `PricebookEntry` custom field would have saved an object, but it scatters the rule set across a standard object shared with every other sales process in the org. The custom catalog keeps the rules in one place that RevOps owns.

5 Entitlement aggregation

5.1 Worked examples

The case the brief calls out, plus one that proves the logic is not hard-coded to two sources.

CASE 1 — BRAND (\$595/MO, INCLUDES CONTACT PACK ×1) + CONTACTS (500) ×1

FEATURE	FROM PLAN	FROM ADD-ONS	ENTITLEMENT
Website hosting	enabled	—	enabled
Listing feeds	enabled	—	enabled
Contacts	500	500	1,000

CASE 2 — ALL IN (\$2,495/MO, INCLUDES CONTACT PACK ×2) + CONTACTS (500) ×3 + COMPETITOR ANALYSIS ×1 + DESIGN REFRESH ×1

FEATURE	FROM PLAN	FROM ADD-ONS	ENTITLEMENT
Website hosting	enabled	—	enabled
Listing feeds	enabled	—	enabled
Competitor analysis	enabled	enabled	enabled — once
Contacts	1,000	1,500	2,500
Design Refresh	—	—	none

D-4 DESIGN DECISION — BINARY VS. METERED IS A FIELD, NOT AN IF

`CPQ_Feature__c.Metering_Type__c` decides whether the reducer sums or ORs. Competitor Analysis arriving from both the plan and an add-on resolves to enabled once because it is Binary; contacts accumulate because they are Metered.

Neither source is privileged — the plan is not a base that add-ons top up. Both contribute through the same path, which is what makes the overlap work in both directions as the brief requires.

D-5 DESIGN DECISION — DESIGN REFRESH ENTITLES NOTHING BECAUSE ITS FEATURE LINK IS NULL

It is a one-off design service, not a capability to gate. It must reach billing and must never reach feature gating.

There is no `if (code == 'ADDON_DESIGN_REFRESH')` anywhere in the code. The absence of a feature link is the mechanism, so any future non-entitling add-on works the same way with no deployment.

5.2 Algorithm

1. Query committed `OpportunityLineItem`s for the won Opportunities where `CPQ_Catalog_Item__c != null`.

2. Skip Setup Fee and One-Time items (R-19).
3. Plan lines → their `CPQ_Plan_Feature__c` rows, contributing `Quantity__c`.
4. Add-on lines → their catalog item's `Feature__c`, contributing `Feature_Quantity_Per_Unit__c × OLI.Quantity`.
5. Reduce into `Map<Id, Map<Id, Decimal>>`, summing where metered and taking presence where binary.
6. Upsert one `Subscription__c` per Opportunity by external key, then insert its entitlements.

Two SOQL queries and two DML statements, regardless of batch size.

6 Integration surface

6.1 Interface choice

OPTION	ASSESSMENT
Custom Apex REST <code>@RestResource</code>	Selected. The caller is a NestJS service speaking HTTP. Typed request/response shapes, explicit status codes, an explicitly versioned URI, and — the deciding factor — the business rules run inside the endpoint where the client cannot route around them.
Standard sObject REST + Composite	Rejected. The platform would assemble line items itself, placing pricing rules in the client. That is the failure mode this project exists to prevent.
Invocable Apex	Rejected. Designed for Flow. Calling it externally via the Actions API gives an awkward wrapper, weaker typing and no natural home for HTTP semantics.
GraphQL / UI API	Rejected. No hook for server-side commercial rules.

6.2 Endpoints

GET	<code>/services/apexrest/cpq/v1/catalog?effectiveDate=2026-09-07</code>
POST	<code>/services/apexrest/cpq/v1/configurations/validate</code>
POST	<code>/services/apexrest/cpq/v1/opportunities/{opportunityId}/commit</code>

GET /catalog returns plans (list price, floor, setup fee, included features and quantities) and add-ons (list price, floor, charge type, mapped feature), filtered to items active on the requested date, defaulting to today. Every item carries its code, so the platform never stores Salesforce Ids. The response includes `catalogVersion` — the maximum `SystemModstamp` across catalog objects — so the platform can cache and cheaply detect staleness.

POST /validate takes a proposed configuration and returns a structured result. No DML, no side effects, safe to call on every keystroke.

LISTING 1 — VALIDATE RESPONSE, TWO VIOLATIONS

```
{
  "correlationId": "b3f1...",
  "valid": false,
  "violations": [
    { "code": "PLAN_BELOW_FLOOR", "line": "PLAN_BRAND",
      "submitted": 425.00, "permitted": 450.00,
      "message": "Brand plan price is below the $450.00 floor." },
    { "code": "ADDON_BELOW_FLOOR", "line": "ADDON_CONTACTS_500",
      "submitted": 70.00, "permitted": 80.00,
      "message": "Contacts (500) price is below the $80.00 floor." }
  ],
  "totals": null
}
```

Structured, not prose — the platform renders its own field-level errors from `line` and `code`, and all violations arrive in one response (R-11).

POST /commit takes the finalised configuration plus an `idempotencyKey`, re-validates from scratch, and writes the plan line, the setup-fee line, the add-on lines and the Opportunity contract fields inside one savepoint. It returns the Opportunity Id, the created line Ids, computed totals (monthly recurring, one-time, first invoice, total contract value) and the correlation ID.

6.3 Status codes and error contract

STATUS	CONDITION
200	Success — including a <code>validate</code> call that returns <code>valid: false</code>
422	Commit refused on business-rule violations; same payload shape as <code>validate</code>
400	Malformed request: bad JSON, missing field, wrong type, unparseable Id
404	Opportunity not found or not accessible — deliberately indistinguishable
409	Opportunity already Closed, or an idempotency key replayed with a different payload
403	The integration principal lacks required object or field access
500	Unexpected — typed error and correlation ID to the caller, stack trace to the log only

D-6 DESIGN DECISION — THE 200/422 ASYMMETRY IS DELIBERATE

`validate` asks whether a configuration is valid, and answering "no" is a successful call, so it returns 200. `commit` asks for a write, and refusing is a failed action, so it returns 422.

Stated explicitly because a client developer will otherwise guess, and guess differently on each endpoint.

Violation codes are a closed, documented set — `PLAN_BELOW_FLOOR`, `ADDON_BELOW_FLOOR`, `SETUP_FEE_ABOVE_CATALOG`, `PLAN_MISSING`, `PLAN_MULTIPLE`, `ADDON_DUPLICATE`, `ADDON_QUANTITY_INVALID`, `UNKNOWN_CATALOG_CODE`, `INACTIVE_CATALOG_ITEM`, `INVALID_CONTRACT_TERM`, `EFFECTIVE_DATE_MISSING`, `NEGATIVE_PRICE`, `PRICE_PRECISION_INVALID` — so the platform can branch on them and localise its own messages. Codes are additive across versions; removing or repurposing one is a breaking change and earns a `/v2`.

6.4 Authentication and rep identity

A Connected App using the OAuth 2.0 **client credentials flow**: server-to-server, no user interaction, no refresh-token lifecycle for the platform to manage. It runs as a named integration user, `svc_revtech_cpq`, provisioned on a **Salesforce Integration license** — API-only, unable to log into the UI, and materially cheaper than a full seat. JWT bearer with a certificate is the alternative if RevTech prefers certificate rotation over secret rotation; both terminate at the same integration user, so the choice does not affect the design.

That user receives exactly one permission set: the six custom objects and the fields this API touches, read on `Product2 / PricebookEntry`, read/write on `Opportunity` and `OpportunityLineItem`, and Apex class access to the three endpoints. No *Modify All Data*, no *View All*.

D-7 DESIGN DECISION — REP IDENTITY IS ATTRIBUTION, NOT AUTHORISATION

Salesforce sees one integration user for every rep in the company. The platform therefore passes `actingRepEmail` in the request body; Salesforce resolves it to an active `User`, rejects it if it does not resolve, and stamps it on `CPQ_Commit__c` and every log row.

Access remains governed entirely by the integration user's permissions and sharing. Treating a client-supplied identity as an authorisation claim would let anyone holding the integration credentials act as anyone. But without capturing it, no discount is traceable to a person — and that traceability is half the reason for enforcing floors at all.

6.5 Data synchronisation

Catalog, Salesforce → platform. Pull with caching. The platform calls `GET /catalog` on boot and on an interval, keyed on `catalogVersion`. The catalog changes a few times a quarter and staleness is low-risk: a stale floor in the client's UI is caught by the server-side check at commit, which is why that check exists. A `CPQ_Catalog_Changed__e` platform event can push invalidation later; it is an optimisation, not a correctness requirement.

Entitlements, Salesforce → downstream. The model is shaped so this becomes a publish rather than a redesign. Because entitlements are already resolved rows with a code, a quantity and an active window, emitting `Customer_Entitlement_Changed__e` on insert is a small addition. Billing is better served by Change Data Capture on `OpportunityLineItem` and `Subscription__c`, or a scheduled pull keyed on the external Id.

D-8 TRADE-OFF — NO CALLOUTS FROM THE CLOSED WON TRIGGER

A callout would make the rep's stage change depend on the availability of two other systems. If billing is down, the deal does not close.

Events decouple that, and a lost event is recoverable by replay in a way that a failed synchronous callout inside a rolled-back transaction is not.

7 Security and observability

7.1 Enforcement on an externally-callable surface

- **Sharing is declared, never inherited.** Apex REST classes run in system context unless told otherwise, so each endpoint class is explicitly `with sharing`. The integration user's record access then genuinely applies.
- **CRUD/FLS enforced by the platform, not by hand.** All queries use `WITH USER_MODE`, and DML runs as `Database.insert(records, AccessLevel.USER_MODE)`. Hand-rolled `isAccessible()` checks drift the moment a field is added; user-mode enforcement does not. `Security.stripInaccessible` shapes any response assembled from records the caller may only partly read.
- **No dynamic SOQL.** Every query is static with bind variables. Where a filter is genuinely dynamic — the catalog's effective-date window — the value is bound, never concatenated. If dynamic SOQL becomes unavoidable, field and object names come from an allow-list, not from the request.
- **Ids are validated before use.** The path parameter passes through `Id.valueOf` inside a try/catch and is checked to be of type `Opportunity` before any query runs. A malformed Id returns a clean `400`, not an unhandled `StringException`.
- **Errors never leak internals.** The caller receives a typed code, a safe message and the correlation ID. The stack trace goes to the log.
- **Input is bounded.** Line counts, string lengths and numeric precision are validated before anything is queried, so a hostile payload cannot burn governor limits on its way to being rejected.

D-9 DESIGN DECISION — "NOT FOUND" AND "NOT PERMITTED" RETURN THE SAME 404

Distinguishing them turns the endpoint into an Id oracle: a caller with valid credentials but no record access could enumerate which Opportunity Ids exist by reading the difference between `403` and `404`.

7.2 Logging

`CPQ_Integration_Log__c` records, per call: correlation ID, endpoint, acting rep, integration user, target Opportunity, outcome, violation codes, duration and timestamp. It is queryable by correlation ID, Opportunity, rep or date range, so support can answer "what happened to this deal?" in one query.

D-10 DESIGN DECISION — LOGS ARE PUBLISHED, NOT INSERTED

A commit that fails rolls back to its savepoint, and a log record inserted inside that transaction rolls back with it. The failures most worth seeing are exactly the ones that erase themselves.

Logs are therefore published as `CPQ_Log_Event__e` platform events with `PublishBehavior = PublishImmediately`, which commits independently of the enclosing transaction. A trigger on the event writes the log row.

Validation failures are logged with their violation codes deliberately. Sub-floor attempts are commercially interesting: the distribution of how far below floor reps are trying to go is the input to the next pricing conversation, and to Q-3.

7.3 Monitoring

The log object feeds a report on error rate, commit volume and top failure reasons, with a dashboard and a reporting snapshot for trend. A sustained error rate above threshold raises an alert. In production these events should forward to the same observability stack Engineering already uses for the NestJS platform rather than building a second one inside Salesforce — one correlation ID spanning both sides is worth more than two good dashboards that do not join. Retention is bounded by a scheduled purge, since an unbounded log object becomes both a storage problem and a slow report.

8 Closed Won automation

A single `after update` trigger on Opportunity delegating to a handler class, written in Apex as the brief requires.

It fires only on the transition *into* Closed Won — the `oldMap` stage is not Closed Won and the new record `IsWon` — so re-saving a won Opportunity does nothing, and an edit to a long-closed deal does not regenerate entitlements.

Idempotency is enforced twice over: by that transition check, and structurally by

`Subscription__c.External_Key__c` being a unique External Id upserted on the Opportunity Id. The transition check handles the common case; the unique key holds if a future entry point bypasses the trigger.

Bulk safety. Queries and DML sit outside all loops — two SOQL and two DML regardless of batch size, correct for a 200-record Data Loader stage update.

D-11 DESIGN DECISION — ENTITLEMENTS COMMIT WITH THE STAGE CHANGE, OR NOT AT ALL

Entitlements are created synchronously in the trigger, so the subscription, the entitlements and the won stage share one transaction. A deal cannot be won with half-built entitlements.

On failure the handler calls `addError` on that specific Opportunity, which fails that record only and lets the other 199 in the batch succeed, and publishes a log event that survives the rollback. It does not swallow the exception.

D-12 TRADE-OFF — WHAT CHANGES WHEN A CALLOUT APPEARS

Synchronous is correct at current volume. If entitlement creation later needs a callout, it moves to a Queueable chained from the trigger and the guarantee weakens from transactional to eventually consistent.

That is worth doing deliberately, with a reconciliation job to match — not discovering it after the fact because someone added a callout to a trigger that used to be safe.

9 Deferred scope

Renewals, amendments, co-terming, proration and cancellation are the next phase, and the model is shaped so they are additions rather than a rewrite. Entitlements already carry an active window and a Superseded status rather than being updated in place; subscriptions already carry term and end date; committed lines already retain their catalog origin.

An amendment therefore becomes a new subscription superseding the old one with a new entitlement generation, and the history of what a customer was entitled to *at a point in time* survives — which is what a billing dispute six months later actually needs.

Approval routing for sub-floor pricing is out of scope by instruction, but it is the most likely first change request once reps start hitting the floor. Because validation returns structured violations rather than throwing, routing a `PLAN_BELOW_FLOOR` to an approval path is a branch in the platform, not a redesign here.

10 Test strategy

Acceptance criteria are expressed as test methods rather than as a document section, so traceability is executable rather than asserted.

TEST METHOD	ASSERTS
<code>catalog_returnsActivePlansAndAddOnsWithCodes</code>	R-1, R-2
<code>catalog_effectiveDateExcludesExpiredItems</code>	R-2
<code>validate_planPriceExactlyAtFloor_passes</code>	R-7 boundary
<code>validate_planPriceOneCentBelowFloor_fails</code>	R-7 boundary
<code>validate_subFloorPlanAndSubFloorAddOn_returnsBothViolations</code>	R-11
<code>validate_setupFeeAboveCatalogAmount_fails</code>	R-9
<code>validate_setupFeeWaivedToZero_passes</code>	R-9
<code>validate_duplicateAddOnLine_fails</code>	R-5
<code>validate_zeroPlansAndTwoPlans_fail</code>	R-4
<code>validate_unknownAndInactiveCodes_returnDistinctViolationCodes</code>	R-2
<code>validate_performsNoDml</code>	no side effects
<code>commit_validConfiguration_createsPlanSetupFeeAndAddOnLines</code>	R-14
<code>commit_subFloorSkippingValidate_createsNothing</code>	R-13, R-14
<code>commit_secondCommit_replacesPriorCpqLinesOnly</code>	R-16
<code>commit_manuallyAddedLine_survivesRecommit</code>	R-16
<code>commit_sameIdempotencyKeyTwice_createsOneSetAndReplaysResult</code>	R-17

TEST METHOD	ASSERTS
commit_closedOpportunity_rejectedWith409	R-15
commit_malformedOpportunityId_returns400NotException	typed errors
closedWon_brandPlusContacts500_entitlesOneThousandContacts	R-18
closedWon_allInWithCompetitorAnalysisAddOn_entitlesOnceNotTwice	R-18 binary
closedWon_designRefresh_billableWithNoEntitlement	R-19
closedWon_setupFee_producesNoEntitlement	R-19
closedWon_bulk200Opportunities_withinGovernorLimits	R-20
closedWon_alreadyWonOpportunityResaved_createsNothing	R-20
closedWon_catalogPriceChangedAfterCommit_entitlementUnaffected	R-3, R-21

Coverage is evidence that the logic is tested, not a number to hit. The boundary cases — exactly at floor, one cent below — the negative paths, and the 200-record bulk case are the ones that matter.

A Appendix — Catalog reference data

A.1 Plans

PLAN	CODE	LIST (MONTHLY)	FLOOR	SETUP FEE
Launch	PLAN_LAUNCH	\$295.00	\$250.00	\$500.00
Brand	PLAN_BRAND	\$595.00	\$450.00	\$1,000.00
All In	PLAN_ALL_IN	\$2,495.00	\$2,000.00	\$2,500.00

A.2 Features and plan inclusion

FEATURE	CODE	METERING	LAUNCH	BRAND	ALL IN
Website hosting	FEAT_WEBSITE_HOSTING	Binary	✓	✓	✓
Listing feeds	FEAT_LISTING_FEEDS	Binary	✓	✓	✓
Competitor analysis	FEAT_COMPETITOR_ANALYSIS	Binary	—	—	✓
Contacts	FEAT_CONTACTS	Metered	—	500	1,000

A.3 Add-ons

PRICES AND FLOORS PER ASSUMPTION A-1, PENDING REVOPS CONFIRMATION (Q-2)

ADD-ON	CODE	FEATURE GRANTED	CHARGE	LIST	FLOOR
Design Refresh	ADDON_DESIGN_REFRESH	none	One-time	\$750.00	\$600.00
Competitor Analysis	ADDON_COMPETITOR_ANALYSIS	FEAT_COMPETITOR_ANALYSIS	Recurring	\$150.00	\$120.00
Contacts (500)	ADDON_CONTACTS_500	FEAT_CONTACTS × 500	Recurring	\$100.00	\$80.00

Prepared by Bruno Romano for the Luxury Presence RevTech Senior Salesforce Developer case study. The accompanying repository contains the data model metadata, the Apex implementation and the test suite described in §10.